Journal of Physics:
Conference Series

PURPOSE-LED PUBLISHING™

PAPER • **OPEN ACCESS**

An improved DDPG algorithm based on evolution-guided transfer in reinforcement learning

To cite this article: Xueqian Bai and Haonian Wang 2024 *J. Phys.: Conf. Ser.* **2711** 012016

View the article online for updates and enhancements.

You may also like

- Deep reinforcement learning-based control of a 1-DOF helicopter: a comparative analysis of classical and modern methods
Serkan Budak, Cemil Sungur and Akif Durdu

- A deep reinforcement learning based metro train operation control optimization considering energy conservation and passenger comfort
Qinyu Tan, Deqiang He, Zheng Sun et al.

- Two degree-of-freedom robotic eye: design, modeling, and learning-based control in foveation and smooth pursuit
Sunil Kumar Rajendran, Qi Wei and Feitian Zhang

This content was downloaded from IP address 70.39.103.3 on 28/03/2026 at 21:04

An improved DDPG algorithm based on evolution-guided transfer in reinforcement learning

Xueqian Bai[1,3], Haonian Wang[2]

[1]College of Information Science and Technology, Beijing University of Chemical Technology, Beijing, 100029, China
[2]College of Management, University of Shanghai for Science and Technology, Shanghai, 200093, China

[3]baicrystal25@gmail.com

Abstract. Deep Reinforcement Learning (DRL) algorithms help agents take actions automatically in sophisticated control tasks. However, it is challenged by sparse reward and long training time for exploration in the application of Deep Neural Network (DNN). Evolutionary Algorithms (EAs), a set of black box optimization techniques, are well applied to single agent real-world problems, not troubled by temporal credit assignment. However, both suffer from large sets of sampled data. To facilitate the research on DRL for a pursuit-evasion game, this paper contributes an innovative policy optimization algorithm, which is named as Evolutionary Algorithm Transfer - Deep Deterministic Policy Gradient (EAT-DDPG). The proposed EAT-DDPG takes parameters transfer into consideration, initializing the DNN of DDPG with the parameters driven by EA. Meanwhile, a diverse set of experiences produced by EA are stored into the replay buffer of DDPG before the EA process is ceased. EAT-DDPG is an improved version of DDPG, aiming at maximizing the reward value of the agent trained by DDPG as much as possible within finite episodes. The experimental environment includes a pursuit-evasion scenario where the evader moves with the fixed policy, and the results show that the agent can explore policy more efficiently with the proposed EAT-DDPG during the learning process.

Keywords: Reinforcement Learning, Evolutionary Algorithm, Parameter Transfer.

1. Introduction

Single-agent autonomous decision-making refers to a situation where an intelligent agent, under a software or hardware system, can accomplish tasks or reach goals without human guidance or external input. Nowadays, such systems have been applied in various fields, including logistics, healthcare, data processing, and electronic gaming. Among them, the pursuit-evasion problem has been a hot issue [1]. It is a category of computational problems and mathematical models typically used to simulate scenarios where one or several pursuers are trying to catch the evader, such as satellite proximate [2], car chases[3], capturing fugitives[4], robotic search[5][6], electronic games[7]-[9], and drone pursuit-evasion[10][11].

The methods of these applications now involve Artificial Fish Swarm Algorithm (AFSA)[12], Newton's method[13] , Asynchronous Advantage Actor-Critic (A3C)[14], Deep Q-Learning Network (DQN)[15], and Deep Deterministic Policy Gradients (DDPG)[16]. The AFSA has issues with low accuracy, more inflection points, and relatively long planning paths. To optimize these shortcomings,

Content from this work may be used under the terms of the Creative Commons Attribution 3.0 licence. Any further distribution of this work must maintain attribution to the author(s) and the title of the work, journal citation and DOI.
Published under licence by IOP Publishing Ltd

an algorithm that combines the enhanced AFSA with continuous segmented Bézier curves was proposed [17]. Although AFSA has wide applications in optimization problems and multi-objective searches, when coping with issues involving dynamic environments and complex state spaces, the A3C algorithm offers a much more flexible and efficient solution. The Asynchronous Advantage Actor-Critic (A3C) algorithm combined with deep neural networks was employed to develop a reactive strategy that learns to exploit behaviors [18]. However, this approach might suffer from the dilemma of local optimization. Nevertheless, it fails to account for the algorithm's ability to adapt to new environments, leading to an inability to showcase its performance benefits when the surroundings change. Moreover, it lacks a comprehensive assessment of various contextual scenarios [19]. An algorithm called Adversary Robust A3C (AR-A3C) was proposed to improve the agent's performance under environments with noise [20]. This makes it more robust against disturbances and enables it to adapt to even more complex scenarios. Unlike A3C, DQN employs experience and target networks to improve data efficiency and network stability. These two innovations make DQN superior in performance of multiple tasks [21][22]. If aiming at optimizing the evader in the game, an anti-pursuit evasion strategy based on Twin-Dueling Double Deep Q-Network (T-D3QN) was proposed, which utilizes two independent Q-networks to estimate the Q-value, resulting in the enhancement of the evader's escape success rate [23]. Furthermore, to avoid sparse rewards, the Monte-Carlo based Deep Q-Networks (MC-DQN) algorithm was adopted [24]. It relies on the Monte-Carlo DRL framework, and is integrated with the DQN algorithm technology. The agent trained with this algorithm has achieved a higher evasion rate in untrained scenarios. Another research based on DQN also aims at successful escape of the evader by introducing a reward mechanism with a time-out strategy and a game environment with an attenuation mechanism for the evader's steering angle. After the improvements, the DQN model effectively elevated the escape probability to the same level as proposed algorithms or even better.

Among various algorithms, Deep Deterministic Policy Gradient (DDPG) stands out as an effective and versatile algorithm for its distinct advantages, particularly when handling problems with continuous actions. It can be adept at dealing with the high-dimensional action spaces commonly encountered in real-world applications like robotics, autonomous vehicles, and complex control systems. The soft target updates employed by DDPG also contribute to more robust and reliable learning, including controlling a car's throttle [25] or a robot's joint angles [26]. Furthermore, the improved version of DDPG has been proposed to enhance learning efficiency. For instance, the training algorithm for the agent's tracking strategy, based on the IL-DDPG algorithm [27], incorporates a quasi-proportional guidance control law to generate effective learning samples. This law serves as a guiding strategy to boost DDPG 's exploration efficiency in the early stages and circumvent excessive unproductive exploration.

While DDPG offers significant advantages in solving complicated problems, it struggles with issues like sparse rewards and slow exploration during the training process. Sparse rewards make it hard for the agent to evaluate the effectiveness of its actions [28], leading to slow learning. Additionally, DDPG's gradient-based nature and experience replay mechanism can trap the agent in local optima and reinforce unproductive behavior, further slowing down the training process. However, Evolutionary Algorithms (EAs) [29] can address these challenges, which operate by conducting a global search in the solution space to identify solutions overlooked by other methods due to sparse rewards. EAs typically maintain a population of solutions, preserving search diversity. This diversity can help the algorithm avoid local optima and potentially locate reward signals in sparse reward environments [30]. Unlike gradient descent algorithms [31], EAs do not need the gradient information. This enables them to be applied to optimization problems that are discontinuous, non-smooth or have high levels of noise. In addition, due to their random search characteristics, EAs are generally insensitive to parameter settings and initial conditions, offering a high level of robustness. With the introduction of EAs, the autonomous decision-making of a single agent is gradually transitioning from simulation to reality with increasing robustness.

Indeed, according to the above literature review, the main methods for single agent decision-making in the pursuit-evasion game include DRL and EAs types, both of which make great contributions to the pursuit-evasion game. With DRL algorithms and their improved ones, the agent’s autonomous decision-making capability is improved, and EAs explore the agent’s further potential ability. However, both

have their own limitations. On the one hand, for DRL algorithms, the agent's observation data will change at any time, which brings the non-stationarity of the environment and has a negative impact on policy update. Besides, over-reliance on expert strategy integration might also constrain the model's learning capability and flexibility. On the other hand, EAs have the limitation of extensive computational resources and time. They may also have slow convergence rates and complex parameter adjustments[32], such as population size, and mutation and crossover rate. Improper parameter settings may lead to a decline in algorithm performance.

In response to the above limitations, our study draws on both DDPG algorithm and the evolutionary algorithm for the pursuit-evasion game. By analyzing their advantages and disadvantages, we provide insight into the integration of them and propose a novel algorithm, *Evolutionary Algorithm Transfer – Deep Deterministic Policy Gradient* (EAT-DDPG), for a single-agent pursuit-evasion game. The proposed EAT-DDPG utilizes EA to generate proper initialized parameters for the actor network of DDPG, enhancing the performance of the pursuer while learning. To sum up, this paper makes main contributions to the current scholarly pursuit-evasion game as follows:

1) To handle challenges of inefficient policy optimization based on data-driven methods in reinforcement learning system, the evolutionary algorithm is introduced to explore effective parameters, which would be transferred to the actor of DDPG. Through proper initialization, the pursuer can form the appropriate policy more efficiently, which helps accomplish tasks within less time.

2) The stopping point is highlighted during the interaction of DDPG training and EA process for lower time-cost and higher convergence value. The rule of stopping is explored and analyzed, aiming at better performance both in the training and test phase.

The remainder of this paper is structured as follows. Section 2 introduces the background knowledge involved in this paper. In section 3, the design principles and implementation process are introduced in detail, as well as the related legend explanations. In section 4, the related contents of the experimental environment, the comparison results, and the analysis of the experimental results are presented. Finally, section 5 draws the conclusion.

2. Background

To introduce background materials regarding EAs and DRL algorithm, this section describes an evolutionary algorithm and DDPG, which will be employed to form the proposed algorithm.

2.1. Evolutionary Algorithm

Evolutionary Algorithms (EAs) are optimization and search algorithms inspired by the process of natural evolution. They operate by maintaining a population of potential solutions to a given problem. Each individual in this population represents a possible solution. A fitness function is used to evaluate the quality of these solutions, measuring how well each solution addresses the problem. Depending on the fitness function, solutions that perform better have a higher probability of being selected as contributors to the next generation. Then two other mechanisms drive the evolution of this population: crossover and mutation. In crossover, solutions referred to as "parents" are combined to produce more "offspring", merging information from both parents. This is done in the hope that the offspring might inherit the best traits from each parent, thus representing a better solution. On the other hand, mutation introduces random changes to these offspring with low probability. This step ensures diversity within the population and helps prevent the algorithm from getting stuck in sub-optimal solutions. As the algorithm progresses, the iterative process of selection, crossover, and mutation optimizes the population, ideally converging to the optimal or near-optimal solution to the problem.

2.2. Deep Deterministic Policy Gradient (DDPG)

Deep Reinforcement Learning (DRL) has demonstrated significant success in a variety of complex tasks, from playing Atari games to mastering Go. However, applying DRL to continuous action spaces presents unique challenges. Traditional methods like Q-learning are well-suited for discrete action spaces, but they cannot effectively scale to continuous domains due to the need to maximize over an

infinite set of actions. To address this challenge, the Deep Deterministic Policy Gradient (DDPG) algorithm was introduced. It is a state of the art, off-policy and model-free algorithm that combines the strengths of Deep Q Networks (DQN) and policy gradient methods, and is specifically designed for environments with continuous action spaces.

It is assumed that there is a scenario of an agent. The transition experience of DDPG is set as tuple (s, a, r, s'), where s is the agent's local observation, a represents its action, r is the reward obtained from the environment, and s' is the observation at the next moment. DDPG employs two neural networks: one is the actor that outputs a deterministic policy, denoted as $\mu(s|\theta^{\mu})$, with parameter θ^{μ}, and the other is the critic that evaluates the Q-value function, denoted as $Q(s, a|\theta^{Q})$, with parameter θ^{Q}. The objective of the critic is to approximate the optimal Q-value function, which is trained using the following loss:

$$L(\theta^{Q}) = E\left[s, a, r, s' \sim D\right][\left(Q(s, a|\theta^{Q}) - \left(r + \gamma Q\left(s', \mu\left(s' \middle|\theta^{\mu}\right)\middle|\theta^{Q}\right)\right)\right)^{2}] \tag{1}$$

Here D represents the experience replay buffer, a crucial component for storing past experiences. γ is the discount factor, emphasizing the significance of future rewards. $\theta^{Q'}$ is the parameter of the target-critic network, which is updated relying on the critic.

The objective of the actor is to select actions that maximize the expected Q-values. Its parameters are updated by ascending the gradient of the expected Q-values:

$$\nabla_{\theta^{\mu}} J = E[\nabla_{a} Q(s, a|\theta^{Q})|_{s\sim D, a=\mu(s)} \nabla_{\theta^{\mu}} \mu(s|\theta^{\mu})] \tag{2}$$

A key innovation of DDPG is the use of target networks for both the actor and critic, guiding the learning with slowly updated target values, thereby stabilizing learning process. θ' is assumed as parameter of target network, and it will be updated through soft updating as follows:

$$\theta' \leftarrow \rho\theta + (1 - \rho)\theta' \tag{3}$$

Here ρ denotes a hyperparameter ranging from zero to one.

3. Methods

In this section, the framework and design process for EAT-DDPG are presented in detail. Firstly, its network architecture in the pursuit-evasion scenario is introduced. Then, its involved evolution-guided exploration process is illustrated. Furthermore, the innovative model parameters transfer is proposed for initializing the Actor network of DDPG algorithm.

3.1. Evolutionary Algorithm Transfer – Deep Deterministic Policy Gradient (EAT-DDPG)

As mentioned in section 2, evolutionary algorithms can help continually generate novel solutions while retaining promising ones, and DDPG is a state of the art deep reinforcement learning off-policy algorithm. However, in real-time scene, due to noise interruption and the complexity of the environment, it is hard to solve problems involving a large number of parameters with optimization techniques of EAs, which are time-consuming. Therefore, it is necessary to introduce DDPG to train the policy only relying on the agent's local observation. Considering the inefficiency of the training process, we introduce the evolution-guided parameters transfer to build an improved DDPG algorithm for pursuit-evasion game.

In the involved pursuit-evasion game, the evader moves with the following velocity:

$$v_e = m_{le} e^{-||p_l - p_e||}(p_l - p_e), v_e \leq v_e^{max} \tag{4}$$

where m_{le} denotes whether evader e has been detected by the learner, while the learn explore policy with the framework of EAT-DDPG. Its general flow proceeds as follows: μ_l, the actor network of DDPG, which is called actor learner in this paper, is initialized alongside a target-actor network, a critic network and a target-critic network. In addition, a population of actors π_{pop} are initialized with random weights

and bias for evolution-guided transfer. Take the ith actor π_{pop}^{i} from π_{pop} as an example. It takes the observation o_i as input and then outputs the action a_i over the timesteps in one episode. The fitness of the ith agent fit_i in this episode is defined as the sum of reward received from each timestep. The actor learner's fitness is computed as the average reward of the latest 50 episodes and the reward setting is presented in Appendix 1.

Fig. 1 illustrates the architecture of EAT-DDPG, which involves mainly two parts, EA and DDPG. As shown in Fig. 1, after the fitness of each actor from π_{pop} is obtained, those with relatively high fitness are regarded as offspring for survival. Crossover and mutation operators are then applied to the population of those with lower fitness after selection (right half part of Fig. 1), creating the next generation of actors for exploration. The details of selection, crossover and mutation operators will be illustrated in section 3.2 later. Two parameter trackers are set in EAT-DDPG, responsible for migrating parameters of an actor network. If

$$fit_l > \min_{i,i\in N} fit_i \tag{5}$$

where fit_l is the fitness of the actor learner and N is the total number of actors in π_{pop}, the parameters of actor learner will be migrated to the actor network in π_{pop} whose fitness is minimum. If

$$fit_l < \max_{i,i\in N} fit_i \tag{6}$$

parameters of the actor network in π_{pop} whose fitness is maximum will be transferred to the actor learner.

A diverse set of new experiences stems from the evaluation process of actors in π_{pop}, and are stored in the replay buffer of DDPG, realizing the flow of information from the evolutionary population to the off-policy learner at the initial training phase. In contrast to the original ERL algorithm regarded as the benchmark in this work [33], we cease the process of EA in advance once the actor learner has learned proper policy to save time. The timing of stopping is illustrated later in section 3.3.

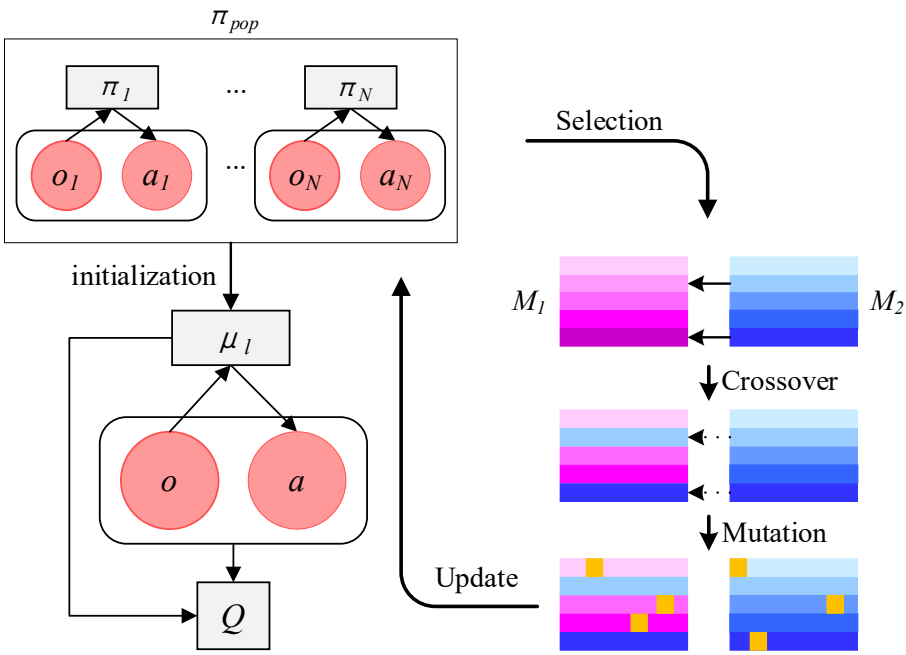

Figure 1. Architecture of EAT-DDPG.

3.2. Evolution-Guided Exploration

Evolutionary algorithms can be implemented to provide robust and powerful adaptive search mechanisms through selection and other genetic operators. Here the genetic algorithm, involving selection, crossover and mutation operators, is applied to explore proper initialization parameters for actor leaner.

Selection: Relying on the fitness of all actors in π_{pop} obtained from the evaluation process, we rank the population to obtain first top e actors as elites where $e < N$. After the tournament selection process, k actors with relatively high fitness will be regarded as offspring from the current generation where $k < N$. Fig. 2 illustrates the framework of the genetic algorithm for initialization exploration. The top is the group of identical matrix variables of actors. Each of them is represented as a set of rectangles, rows of the matrix. Two actors, neither elite nor offspring, are selected in an iteration, and then perturbed through crossover and mutation operations. All types of variables are operated.

Crossover: Before this operation, the parameters of one elite and an actor from offspring are first migrated to the two selected actors, which are involved in the crossover operation. For M_1 and M_2, the two-dimensional variables of selected actors π_1 and π_2 in π_{pop}, we replace a row of M_1 or M_2 for N_cr times with the principle:

$$\begin{cases} M_1^r = M_2^r, if\ ran < 0.5 \\ M_2^r = M_1^r, if\ ran > 0.5 \end{cases} \tag{7}$$

Here M is the weight or bias matrix, r is its rth row and ran is a random number from 0 to 1. For example, in Fig. 2, M_1's second and fifth rows are replaced with that of M_2 through crossover.

Mutation: This part is for all actors which are not elites in π_{pop}. For each variable M of an actor, one of its numbers is operated during $0.1 * |M|$ iterations. In each iteration, three mutation choices are set as follows:

$$\begin{cases} M_{i,j} = M_{i,j} * N\left(0{,}100 * mut_{strength}\right), if\ ran < pro_{super} \\ M_{i,j} = N(0{,}1), if\ ran < pro_{reset} \\ M_{i,j} = M_{i,j} * N\left(0, mut_{strength}\right), if\ ran > pro_{reset} \end{cases} \tag{8}$$

Here pro_{super} and pro_{reset} are the probabilities of the first two operation choices and are set to 0.05 and 0.07 respectively. $mut_{strength}$ is the strength of mutation and is set to 0.1 in the experiment. Mutation point $M_{i,j}$ is the *i*th row and *j*th column of M, represented by the yellow part in Fig. 2.

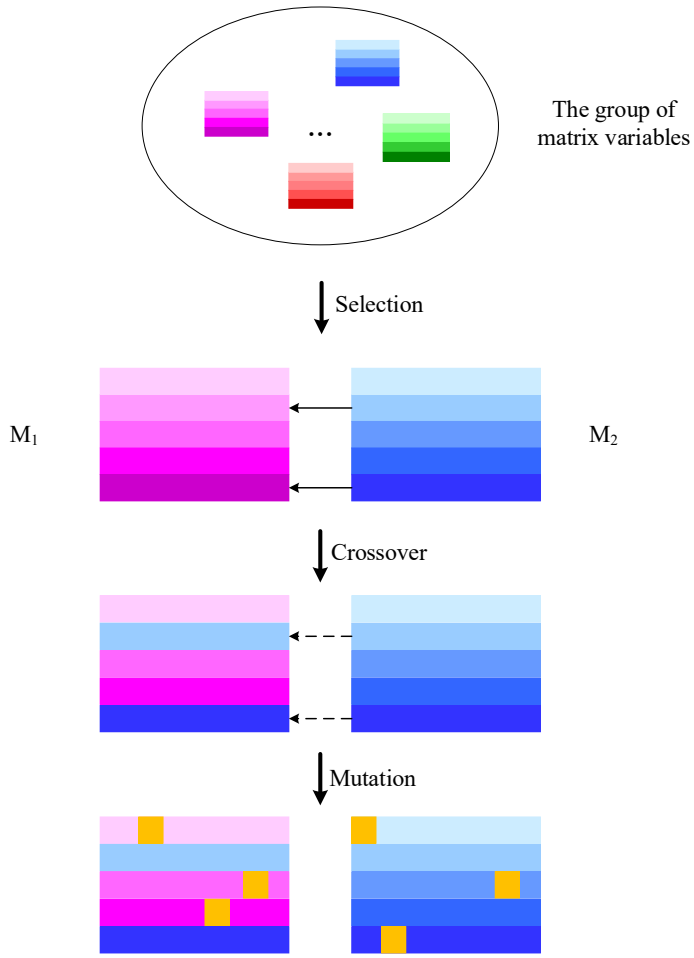

Figure 2. Framework of the genetic algorithm for initialization exploration.

3.3. Model Parameters Transfer

EAT-DDPG aims to offer a proper initialization for actor learner in the training phase. The timing of stopping EA process is essential for further training and this part analyzes the possibility of different points of stopping. According to the reward setting of the learner in Appendix 1, the sum of reward until the current time of one episode will be less than zero if the learner has not reached its target, which means the distance between them has never been less than the threshold distance. If the maximum of their distance is estimated as 2 and the learner fails in this round, depending on the value of episode length, the overall reward will be less than minus 3. Therefore, once the learner reaches the evader, getting the positive reward 10, the overall reward of this round must be larger than zero. We use zero as a dividing line, and perform experiments on the values of four stopping points, 0.5, 1, 1.5, 2. This threshold reward is denoted as r_{thre} later, and the time-cost of the training phase and performance of the learner are also illustrated in the experiment part.

Algorithm 1: Evolutionary Algorithm Transfer – Deep Deterministic Policy Gradient (EAT-DDPG)

Initialize actor learner μ_l, critic Q_l, target-actor μ_l' and target-critic Q_l' with parameters θ^{μ}, θ^{Q}, $\theta^{\mu'}$ and $\theta^{Q'}$, respectively
Initialize a random process P for policy exploration and a random number generator $ran\sim(0,1)$
Initialize a replay buffer D with size of S_D for experience storage
Initialize a population of actors π_{pop} with parameters θ_{π} for initialization exploration

for generation = 1 to E_g **do**
  if $avr < r_{thre}$ **then**
    Initialize a list F for fitness storage
      for actor $\pi_i \in \pi_{pop}$ **do**
        Receive initial state o_i
        Execute actions $a_i = \pi(o_i)$, observe reward r_i and new state o_i'
        Store r_i in F and (o_i, a_i, r_i, o_i') in D
      end for
      Rank the values in F and select the first e actors as elites π_{eli} where $e < N$
      Utilize tournament selection to obtain first k offspring π_{off} where $k < N$
      for every two matrix M_1 and $M_2 \notin \theta_{eli}$ or θ_{off} **do**
        Migrate parameters from elite or offspring to M_1 and M_2
        Update M_1 or M_2 with the principle Eq. (3-4)
      end for
    for actor $\pi_i \notin \pi_{eli}$ or π_{off} **do**
      for $M_i \in \theta_i$ **do**
        if $ran < 0.9$ **do**
          for p = 1 to $0.1 * |M|$ **do**
            update M_i with the principle Eq. (3-5)
          end for
        end if
      end for
    end for
  end if
  Receive initial state o_l
  for t = 1 to E_l **do**
    Execute actions $a_l = \mu_l(o_l)$, observe r_l and new state o_l'
    Store (o_l, a_l, r_l, o_l') in D
    $o_l \rightarrow o_l'$
  end for
  Sample a random minibatch of S_b samples $\left(x^m, a^m, r^m, x'^m\right)$ from D
  Set $y^m = r^m + \gamma Q^{\mu'}(o', a')|_{a_l'=\mu_l'\left(o_l'^m\right)}$
  Update critic by minimizing the loss:

$$L(\theta^Q) = \frac{1}{S_b}\sum_m \left(y^m - Q^\mu(o^m, a^m)\right)^2|_{a_l=\mu_l(o_l^m)}$$

  Update actor by minimizing the loss:

$$L(\theta^\mu) = -\frac{1}{S}\sum_m \left(Q^\mu(o^m, a^m)\right)$$

  Update target networks' parameters:

$$\theta^{Q'} \leftarrow \rho\theta^Q + (1-\rho)\theta^{Q'}$$
$$\theta^{\mu'} \leftarrow \rho\theta^\mu + (1-\rho)\theta^{\mu'}$$

    Update π_{pop} or μ_l depending on condition Eq. (3-2)-(3-3)
  end for

4. Experimental Results

To analyze the performances of the proposed EAT-DDPG, this section first introduced the experiment setting for the pursuit-evasion game. Besides, we compare the performance of EAT-DDPG with a standard evolutionary algorithm (EA), DDPG and original ERL algorithm as benchmark.

4.1. Experiment Setting

The experiments are performed in a two-dimensional environment with Gaussian noise, obeying the distribution $N(0,1)$. The training hyperparameters and environment parameters are shown in Table 1 and

Table 2 respectively. The experiments are performed based on PyTorch.

4.2. Environment

This paper employs a two-dimensional environment which is based on continuous observation space and action space, and the physical items including air resistance, collision forces and drive forces. The action space for agent trained includes “Still”, “North”, “South”, “West” and “East”, so the direction of acceleration can be represented by a five-dimensional probability distribution, with the sum of the five dimensions being one. In this pursuit-evasion game, as shown in Fig. 3, the evader moves with a fixed policy, and the learner is trained to get as close as possible to its target within 25 unit-time. The closer the learner to the moving evader, the greater its reward value. When it reaches the threshold distance of the evader, an additional reward value will be obtained, marking the completion of the task. In this way, the success rate is calculated based on the proportion of the completed tasks over the total episodes.

The experiments adopt EAT-DDPG, DDPG, EA and ERL benchmark in the pursuit-evasion scenario for comparisons respectively. The initial position coordinates of the learner range from 0.7 to 0.9, and that of the evader ranges from -0.9 to -0.5. The initial velocity and acceleration of both agents are set as 0.

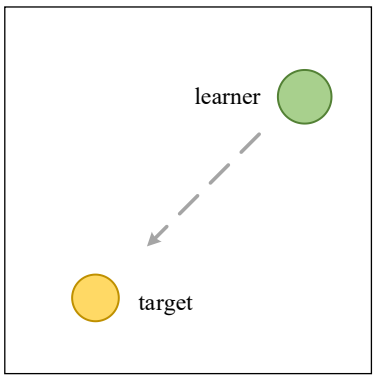

Figure 3. Illustration of pursuit-evasion scenario.

Table 1. Training value information.

Donation	Explanation	Value
S_b	batch size	1024
S_D	replay buffer size	1000000
lr_{actor}	actor network learning rate	0.001
lr_{critic}	critic network learning rate	0.001
γ	discount factor	0.95
ρ	soft-update coefficient	0.01
hl_1	neuron number in hidden layer 1	64
hl_2	neuron number in hidden layer 2	64

Table 1. (continued).

E_g	training episode number	11000
E_{test}	test episode number	2000
E_l	episode length	25
e	elite number	1
k	offspring number	2
r_{thre}	threshold distance	0.1

Table 2. Environment parameters.

Experiment Parameters	Value
acceleration of learner	10
acceleration of evader	5
max velocity of learner	1.0
max velocity of evader	0.6
learner size	0.075
evader size	0.05

4.3. Performance Comparison in Pursuit-evasion Scenario

There are two agents in this scenario, including one learner and one evader. The performance comparison consists of two parts. For section *4.3.1*, it shows and analyzes the average reward learning curve of the learner. For section *4.3.2*, the time-cost of training phase and performances while testing are analyzed empirically. For section *4.3.3*, reward curves and test results of four stopping points are presented meticulously.

4.3.1. Average Reward of Training Phase

Fig. 4 compares the average reward, the average of the latest 50 episodes' reward, of DDPG, EA, EAT-DDPG and ERL in the training phase. The training hyperparameters of them are all the same as shown in Table 1. The initial values of the rewards with the four algorithms are around -6. Then values of EAT-DDPG, DDPG and ERL rise with training and eventually converge. Compared with EA, the other three reward curves have climbed to higher values along with the episodes rising. Because of the cumulation of effective experiences, DDPG rises faster than EA and converges to about 15. For ERL, there is no evident increase at the initial phase, but its reward rises significantly after 3,000 episodes with the effort of evolutionary exploration and data experience. EAT-DDPG trains the learner with the help of the EA initialization exploration, transferring parameters to actor network of DDPG. Its reward rises much faster than other three algorithms, and converges to about 17 more rapidly, which is the best performance over the compared algorithms.

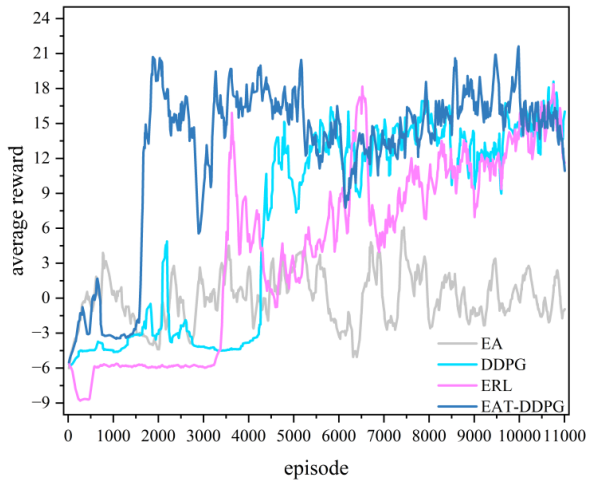

Figure 4. Average reward curve comparison in the training phase.

4.3.2. Time-cost of Training Phase and Test Results

To verify the efficiency of our innovative algorithm, the time-cost of four algorithms in the training phase and their test results are compared in Table 3. The value of threshold reward $r_{thre} = 1.0$ here. EA is relatively time-consuming due to the process of evaluation. According to the column Time in Table 3, the values are much larger with the involvement of EA process. DDPG is the most time-saving algorithm among them, but its success rate in the test phase is smaller than that of EAT-DDPG, around 82% and 87% respectively. The original ERL algorithm outperforms both EA and DDPG in terms of performance, but it comes at the cost of a significantly longer training time, approximately 10 times that of DDPG and even larger than EA. In essence, EA does not contribute to time efficiency. However, when coupled with its innovative initialization exploration, EAT-DDPG emerges as the top performer during the testing phase, without imposing substantial time constraints during the training period.

Table 3. Time-cost and test results of four algorithms.

Algorithm	Time/s	Success Rate
EAT-DDPG	452.79	87.40%
EA	3286.24	79.01%
DDPG	358.09	80.20%
ERL	3598.96	83.65%

4.3.3. Time-cost and Testing Results of Parameters Transfer

To explore the proper stopping point of EA process, the performances with different values of threshold reward r_{thre} are compared in this part. The performances of different stopping points, 0.5, 1.0, 1.5, 2.0, are shown in Fig. 5. According to the figure, the performance of $r_{thre} = 1.0$ is better than that of others, with the reward rising faster and converging to about 16. Its success rate is also the highest, reaching 87.4% shown in Table 4. The smaller the value of r_{thre}, the shorter the time-cost, but there is no evident divergence between different stopping points. The test results of first three stopping points are better than that of other algorithms in Table 3 except for $r_{thre} = 2.0$, indicating that the stopping point should not be late. The experimental result illustrates the reasonableness of exploring stopping points at the early phase of training while there is not much difference in the training time.

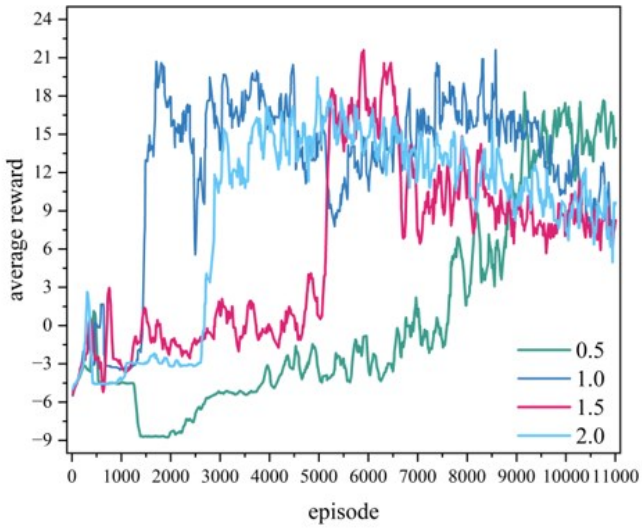

Figure 5. Average reward curve comparison of different stopping points.

Table 4. Time-cost and test results of four stopping points.

r_{thre}	Time/s	Success Rate
0.5	440.42	84.35%
1	452.79	87.40%
1.5	462.43	86.65%
2	487.28	82.45%

5. Conclusion

In this paper, we propose to optimize DDPG by introducing EA algorithm to offer actor network a proper initialization. The experiments are performed on a two-dimensional environment, where EAT-DDPG is compared with its baselines, DDPG, EA and ERL. From experimental results, the reward of our approach EAT-DDPG rises and converges much faster than the other three, with not much more time-cost compared to the minimum of the training time, also reaching the highest task success rate during the test phase. To sum up, EAT-DDPG has shown significant improvements over baselines not only in the performance and time-cost of the training phase but also the test result.

References

[1] R. Isaacs, "Differential Games: A Mathematical Theory with Applications to Warfare and Pursuit, Control and Optimization," Courier Corporation, 1999.

[2] D. Ye, M. Shi, and Z. Sun, "Satellite proximate pursuit-evasion game with different thrust configurations," Aerospace Science and Technology, vol. 99, p. 105715, Apr. 2020, doi: 10.1016/j.ast.2020.105715.

[3] I. Exarchos and P. Tsiotras, "An asymmetric version of the two car pursuit-evasion game," 53rd IEEE Conference on Decision and Control (CDC), pp. 4272-4277. Dec. 01, 2014.

[4] S. Li, X. Wang, Y. Zhang, W. Xue, J. Černý, and B. An, "Solving Large-Scale Pursuit-Evasion Games Using Pre-trained Strategies," Proceedings of the AAAI Conference on Artificial Intelligence, vol. 37, no. 10, pp. 11586–11594, Jun. 2023.

[5] A. Bajcsy, A. Loquercio, A. Kumar, and J. Malik, "Learning Vision-based Pursuit-Evasion Robot Policies," arXiv.org, Aug. 30, 2023, doi: 10.48550/arXiv.2308.16185.

[6] S. Sachs, S. M. LaValle, and S. Rajko, "Visibility-Based Pursuit-Evasion in an Unknown Planar Environment," The International Journal of Robotics Research, vol. 23, no. 1, pp. 3–26, Jan. 2004, doi: 10.1177/0278364904039610.
[7] M. Wang, L. Wang, and T. Yue, "An Application of Continuous Deep Reinforcement Learning Approach to Pursuit-Evasion Differential Game," 2019 IEEE 3rd Information Technology, Networking, Electronic and Automation Control Conference (ITNEC), pp. 1150-1156, Mar. 01, 2019.
[8] X. Huang, "StarCraft adversary-agent challenge for pursuit–evasion game," Journal of the Franklin Institute, Aug. 2023, doi: 10.1016/j.jfranklin.2023.08.032.
[9] D. W. Oyler, P. T. Kabamba, and A. R. Girard, "Pursuit–evasion games in the presence of obstacles," Automatica, vol. 65, pp. 1–11, Mar. 2016, doi: 10.1016/j.automatica.2015.11.018.
[10] Efe Camci and Erdal Kayacan, "Game of drones: UAV pursuit-evasion game with type-2 fuzzy logic controllers tuned by reinforcement learning," 2016 IEEE International Conference on Fuzzy Systems (FUZZ-IEEE), pp. 618-625, Jul. 2016.
[11] L. Anderson and S. Senapathy, "On Solving the 2-Dimensional Greedy Shooter Problem for UAVs," arXiv.org, Nov. 02, 2019, doi: 10.48550/arXiv.1911.01419.
[12] L. Xiao, "An Optimizing Method Based on Autonomous Animats: Fish-swarm Algorithm," Systems engineering-theory & practice, vol. 22, Nov. 11, pp.32-38, 2002, doi: 10.12011/1000-6788(2002)11-32.
[13] Z. Li, H. Zhu, Z. Yang, and Y. Luo, "A dimension-reduction solution of free-time differential games for spacecraft pursuit-evasion," Acta Astronautica, vol. 163, pp. 201–210, Oct. 2019, doi: 10.1016/j.actaastro.2019.01.011.
[14] V. Mnih et al., "Asynchronous Methods for Deep Reinforcement Learning," 33rd International Conference on Machine Learning (ICML), pp. 1928-1937, Jun. 11, 2016.
[15] V. Mnih et al., "Human-level control through deep reinforcement learning," Nature, vol. 518, no. 7540, pp. 529–533, Feb. 2015, doi: 10.1038/nature14236.
[16] T. P. Lillicrap et al., "Continuous control with deep reinforcement learning," arXiv.org, 2015, doi: 10.48550/arXiv.1509.02971.
[17] F.-F. Li, Y. Du, and K.-J. Jia, "Path planning and smoothing of mobile robot based on improved artificial fish swarm algorithm," Scientific Reports, vol. 12, no. 1, Jan. 2022, doi: 10.1038/s41598-021-04506-y.
[18] B. Vlahov, E. Squires, L. Strickland, and C. Pippin, "On Developing a UAV Pursuit-Evasion Policy Using Reinforcement Learning," 2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA), pp. 859-864, Dec. 01, 2018.
[19] B. Li, H. Zhang, P. He, G. Wang, K. Yue, and E. Neretin, "Hierarchical Maneuver Decision Method Based on PG-Option for UAV Pursuit-Evasion Game," Drones, vol. 7, no. 7, p. 449, Jul. 2023, doi:10.3390/drones7070449.
[20] Z. Gu, Z. Jia, and H. Choset, "Adversary A3C for Robust Reinforcement Learning," arXiv.org, Dec. 01, 2019, doi: 10.48550/arXiv.1912.00330.
[21] J. Zhu, W. Zou, and Z. Zhu, "Learning Evasion Strategy in Pursuit-Evasion by Deep Q-network," 2018 24th International Conference on Pattern Recognition (ICPR), pp. 67-72, Aug. 01, 2018.
[22] C. Xu, Y. Zhang, W. Wang, and L. Dong, "Pursuit and Evasion Strategy of a Differential Game Based on Deep Reinforcement Learning," Frontiers in Bioengineering and Biotechnology, vol. 10, p. 827408, Mar. 2022, doi: 10.3389/fbioe.2022.827408.
[23] X. Liang, Y. Yang, Q. Wang, W. Wang, and W. Han, "Research on Anti-pursuit Evasion Strategy of Unmanned Surface Vehicle Based on T-D3QN," 2022 5th International Conference on Intelligent Autonomous Systems (ICoIAS), pp. 172-178, Sep. 01, 2022.
[24] K. Chen, J. Lei, and B. Li, "The Pursuit-Evasion Game Strategy of High-Speed Aircraft Based on Monte-Carlo Deep Reinforcement Learning," 2021 5th Chinese Conference on Swarm Intelligence and Cooperative Contro(CCSICC), pp. 1616–1627, Jul. 2022.

[25] V. S. Dubey, R. Kasad, and K. Agrawal, "Autonomous Braking and Throttle System: A Deep Reinforcement Learning Approach for Naturalistic Driving," arXiv.org, Aug. 15, 2020, doi: 10.48550/arXiv.2008.06696.
[26] Y. Li, Z. Chen, C. Wu, H. Mao, and P. Sun, "A Hierarchical Framework for Quadruped Robots Gait Planning Based on DDPG," Biomimetics, vol. 8, no. 5, p. 382, Sep. 2023, doi: 10.3390/biomimetics8050382.
[27] X. Fu, J. Zhu, Z. Wei, H. Wang, and S. Li, "A UAV Pursuit-Evasion Strategy Based on DDPG and Imitation Learning," International Journal of Aerospace Engineering, vol. 2022, p. e3139610, May 2022, doi: 10.1155/2022/3139610.
[28] R. S. Sutton and A. G. Barto, "Reinforcement Learning, second edition: An Introduction," MIT Press, 2018.
[29] D. B. Fogel, "Evolutionary computation: toward a new philosophy of machine intelligence," John Wiley & Sons, 2006.
[30] W. M. Spears, K. A. Jong, T. Bäck, D. B. Fogel, and H. Garis, "An overview of evolutionary computation," European Conference on Machine Learning(ECML), pp. 442–459, 1993.
[31] S. Ruder, "An overview of gradient descent optimization algorithms," arXiv.org, 2016, doi: 10.48550/arXiv.1609.04747
[32] A. E. Eiben, R. Hinterding, and Z. Michalewicz, "Parameter control in evolutionary algorithms," IEEE Transactions on Evolutionary Computation, vol. 3, no. 2, pp. 124–141, Jul.1999.
[33] S. Khadka and K. Tumer, "Evolution-Guided Policy Gradient in Reinforcement Learning," 32nd Conference on Neural Information Processing Systems (NIPS), pp. 1188-1200, Nov. 2018.

Appendix 1 Description of Reward Setting

The basic principle of the reward design for agent *i* is based on the distance between the evader and it, and a higher value will be given when the distance is less than the set reaching distance.

The reward of agent *i* is designed as:

$$reward -= 0.1\sqrt{P_i^2 - P_{tar}^2}, if \sqrt{P_i^2 - P_{tar}^2} > capt \tag{9}$$

$$reward += 10, if \sqrt{P_i^2 - P_{tar}^2} \leq capt \tag{10}$$

where $\sqrt{P_i^2 - P_{tar}^2}$ represents the distance between agent *i* and the evader, and $capt$ is the reaching distance. When the distance between them is larger than $capt$, agent *i* will be rewarded with a value negatively correlated to the distance. Besides, once the distance between them is less than $capt$, it will obtain an extra reward, 10.